

EX.NO: 1.1**ARRAY IMPLEMENTATION OF LIST ADT****Aim:**

To write a C Program to implement list and its operations using array.

Algorithm:

Step 1: Declare and initialize the values for the variables. Declare the maximum size of the array.

Step 2: Create list

- Get the number of elements in the list from the user. Get the list elements from the user and update in the array.

Step 3: Insert

- Get the element to be inserted and the position of that element from the user. Insert the given element in the given position as follows
- If the element is already present in that position then shift the existing elements to the right and update the position with new element.
- If the given position is empty then update that position with the new element.

Step 4: Search

- Get the search element from the user.
- Check whether the given element is present in the array
- If the search element is present, it will return the position of that element.
- If the search element is not present, then it will display the element is not found.

Step 5: Delete

- Get the position of the element to be deleted from the user.
- Delete the element in the given position and shift the next elements to the left.

Step 6: Display

- Print all the elements present in the array.

Program:

```
#include<stdio.h>
#define MAX_SIZE 100
void createArray(int arr[], int *size);
void insertElement(int arr[], int *size, int element, int position);
int searchElement(int arr[], int size, int element);
void deleteElement(int arr[], int *size, int position);
void displayArray(int arr[], int size);
int main() {
    int arr[MAX_SIZE];
    int size = 0;
    int choice, element, position, result;
    while(1) {
        printf("\nArray Operation Menu:\n");
        printf("1. Create Array\n");
        printf("2. Insert Element\n");
        printf("3. Search Element\n");
        printf("4. Delete Element\n");
        printf("5. Display Array\n");
```

```
printf("6. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice) {
    case 1:
        createArray(arr, &size);
        break;
    case 2:
        printf("Enter element to insert: ");
        scanf("%d", &element);
        printf("Enter position to insert (0 to %d): ", size);
        scanf("%d", &position);
        insertElement(arr, &size, element, position);
        break;
    case 3:
        printf("Enter element to search: ");
        scanf("%d", &element);
        result = searchElement(arr, size, element);
        if(result != -1)
            printf("Element found at position: %d\n", result);
        else
            printf("Element not found in the array\n");
        break;
    case 4:
        printf("Enter position to delete (0 to %d): ", size-1);
        scanf("%d", &position);
        deleteElement(arr, &size, position);
        break;
    case 5:
        displayArray(arr, size);
        break;
    case 6:
        return 0;
    default:
        printf("Invalid choice! Please enter a valid option.\n");
}
}
return 0;
}

void createArray(int arr[], int *size) {
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    if(n > MAX_SIZE) {
        printf("Error: Number of elements exceeds the maximum array size.\n");
        return;
    }
    printf("Enter %d elements: ", n);
    for(i=0; i < n; i++) {
        scanf("%d", &arr[i]);
        *size = n;
        printf("Array created successfully.\n");
    }
}

void insertElement(int arr[], int *size, int element, int position) {
    int i;
    if(*size >= MAX_SIZE) {
        printf("Error: Array is full. Cannot insert element.\n");
        return;
    }
    if((position < 0) || (position > *size)) {
```

```
        printf("Error: Invalid position.\n");
        return;
    }
    for(i=0; i < size; i++) {
        arr[i] = arr[i - 1];
    }
    arr[position] = element;
    (*size)++;
    printf("Element inserted successfully.\n");
}
int searchElement(int arr[], int *size, int position) {
    int i;
    for(i = 0; i < size; i++) {
        if(arr[i] == element) {
            return i;
        }
    }
    return -1;
}
void deleteElement(int arr[], int *size, int position) {
    int i;
    if((position < 0) || (position >= *size)) {
        printf("Error: Invalid position.\n");
        return;
    }
    for(i = position; i < *size - 1; i++) {
        arr[i] = arr[i + 1];
    }
    (*size) --;
    printf("Element deleted successfully.\n");
}
void displayArray(int arr[], int size) {
    int i;
    if(size == 0) {
        printf("Array is empty.\n");
        return;
    }
    printf("Array elements: ");
    for(i=0; i < size; i++) {
        printf("%d", arr[i]);
    }
    printf("\n");
}
```

Output:

Array Operations Menu:

1. Create Array
2. Insert Element
3. Search Element
4. Delete Element
5. Display Array
6. Exit

Enter your choice: 1

Enter the number of elements: 5

Enter 5 elements: 1 2 3 4 5

Array created successfully.

Array Operations Menu:

1. Create Array
2. Insert Element
3. Search Element
4. Delete Element

```

5. Display Array
6. Exit
Enter your choice: 2
Enter element to insert: 10
Enter position to insert (0 to 5): 2
Element inserted successfully.
Array Operations Menu:
1. Create Array
2. Insert Element
3. Search Element
4. Delete Element
5. Display Array
6. Exit
Enter your choice: 5
Array elements: 1 2 10 3 4 5
Array Operations Menu:
1. Create Array
2. Insert Element
3. Search Element
4. Delete Element
5. Display Array
6. Exit
Enter your choice: 3
Enter the element to search: 10
Element found at position: 2
Array Operations Menu:
1. Create Array
2. Insert Element
3. Search Element
4. Delete Element
5. Display Array
6. Exit
Enter your choice: 4
Enter position to delete (0 to 5): 2
Element deleted successfully.
Array Operations Menu:
1. Create Array
2. Insert Element
3. Search Element
4. Delete Element
5. Display Array
6. Exit
Enter your choice: 5
Array elements: 1 2 3 4 5

```

Result:

Thus, the C program to implement list ADT using array has been executed and verified successfully and satisfied the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.1	2	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 1.2**LINKED LIST IMPLEMENTATION OF LIST ADT****Aim:**

To write a C Program to implement the list and its operations using linked list.

Algorithm:

Step 1: Declare and initialize the values for the variables. Declare the structure for linked list node.

Step 2: Create node

- Get the value from the user and allocate the memory for the new node dynamically and update the value in the data field of the new node and update its next pointer field to NULL.

Step 3: Insert at end

- Get the value from the user and allocate the memory for the new node dynamically and update the value in the data field of the new node and update its next pointer field to NULL.
- Traverse list to the end node and link the next pointer field of the last node to the newnode.

Step 4: Insert at Position

- Get the value and position from the user and allocate the memory for the new node dynamically and update the value in the data field of the new node.
- Traverse list to the given position and link the next pointer field of the node in the given position to the newnode.
- Link the next node of the given position to the newnode.

Step 4: Search

- Get the search element from the user.
- Check whether the given element is present in the list
- If the search element is present, it will return the position of that element.
- If the search element is not present, then it will display the element is not found.

Step 5: Delete

- Get the position of the element to be deleted from the user.
- Traverse the lists to the given position link the previous node to the next node of the given position.
- Free the memory allocated for that node.

Step 6: Display

- Print all the elements present in the list.

Program:

```
#include<stdio.h>
#include<stdlib.h>
//Define the structure for a node
struct Node{
    int data;
    struct Node* next;
};
//Function prototypes
struct Node* createNode(int data);
void insertAtEnd(struct Node** head, int data);
void insertAtPosition(struct Node** head, int data, int position);
int searchElement(struct Node** head, int data);
void deleteElement(struct Node** head, int position);
```

```
void displayList(struct Node* head);
int main() {
    struct Node* head = NULL;
    int choice, data, position, result;
    while(1) {
        printf("\nSingly Linked List Operations Menu:\n");
        printf("1. Insert at End\n");
        printf("2. Insert at Position\n");
        printf("3. Search Element\n");
        printf("4. Delete Element\n");
        printf("5. Display List\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                printf("Enter data to insert at end: ");
                scanf("%d", &data);
                insertAtEnd(&head, data);
                break;
            case 2:
                printf("Enter data to insert: ");
                scanf("%d", &data);
                printf("Enter position to insert (0 – based index): ");
                scanf("%d", &position);
                insertAtPosition(&head, data, position);
                break;
            case 3:
                printf("Enter element to search: ");
                scanf("%d", &data);
                result = searchElement(&head, data);
                if(result != -1)
                    printf("Element found at position: %d\n", result);
                else
                    printf("Element not found in the list\n");
                break;
            case 4:
                printf("Enter position to delete (0 – based index): ");
                scanf("%d", &position);
                deleteElement(&head, position);
                break;
            case 5:
                displayList(head);
                break;
            case 6:
                //Free memory allocated for the linked list before exit
                while(head != NULL) {
                    struct Node* temp = head;
                    head = head->next;
                    free(temp);
                }
                return 0;
            default:
                printf("Invalid choice! Please enter a valid option.\n");
        }
    }
    return 0;
}

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if(newNode == NULL) {
```

```
        fprintf(stderr, "Memory allocation failed\n");
        exit(EXIT_FAILURE);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
// Function to insert a note at the end of the list
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if(*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while(temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
//Function to insert a node at a specified position
void insertAtPosition(struct Node** head, int data, int position)
{
    if(position == 0) {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    struct Node* current = *head;
    struct Node* previous = NULL;
    int currentPos = 0;
    while((current != NULL) && (currentPos < position)) {
        previous = current;
        current = current->next;
        currentPos++;
    }
    if(currentPos != position) {
        printf("Error: Invalid position.\n");
        free(newNode);
        return;
    }
    newNode->next = current;
    previous->next = newNode;
}
//Function to search for an element in the list
int searchElement(struct Node **head, int data) {
    struct Node* temp = *head;
    int position = 0;
    while(temp != NULL) {
        if(temp->data == data) {
            return position;
        }
        temp = temp->next;
        position++;
    }
    return -1;
}

// Function to delete a node at a specified position
void deleteElement(struct Node** head, int position) {
    if (*head == NULL) {
        printf("Error: List is empty.\n");
    }
}
```

```
        return;
    }
    if (position < 0) {
        printf("Error: Invalid position.\n");
        return;
    }
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);

        return;
    }
    struct Node* previous = NULL;
    int currentPos = 0;
    while (temp != NULL && currentPos < position) {
        previous = temp;
        temp = temp->next;
        currentPos++;
    }
    if (temp == NULL) {
        printf("Error: Invalid position.\n");
        return;
    }
    previous->next = temp->next;
    free(temp);
}
// Function to display the list
void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked List: ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Output:

Singly Linked List Operations Menu:

1. Insert at End
2. Insert at Position
3. Search Element
4. Delete Element
5. Display List
6. Exit

Enter your choice: 1

Enter data to insert at end: 10

Singly Linked List Operations Menu:

1. Insert at End
2. Insert at Position
3. Search Element
4. Delete Element
5. Display List
6. Exit

Enter your choice: 2
 Enter data to insert: 20
 Enter position to insert (0-based index): 1
 Singly Linked List Operations Menu:
 1. Insert at End
 2. Insert at Position
 3. Search Element
 4. Delete Element
 5. Display List
 6. Exit
 Enter your choice: 5
 Linked List: 10 -> 20 -> NULL
 Singly Linked List Operations Menu:
 1. Insert at End
 2. Insert at Position
 3. Search Element
 4. Delete Element
 5. Display List
 6. Exit
 Enter your choice: 3
 Enter element to search: 20
 Element found at position: 1
 Singly Linked List Operations Menu: 1.
 1. Insert at End
 2. Insert at Position
 3. Search Element
 4. Delete Element
 5. Display List
 6. Exit
 Enter your choice: 4
 Enter position to delete (0-based index): 1
 Singly Linked List Operations Menu:
 1. Insert at End
 2. Insert at Position
 3. Search Element
 4. Delete Element
 5. Display List
 6. Exit
 Enter your choice: 5
 Linked List: 10 -> NULL

Result:

Thus the C program to perform linked list implementation of list ADTs has been successfully executed and output is verified and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.1	2	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 2**IMPLEMENTATION OF CIRCULAR LIST ADT****Aim:**

To write a C Program to implement circular linked list and its various operations.

Algorithm:

Step 1: Declare and initialize the values for the variables. Declare the structure for circular linked list node.

Step 2: Insert at end

- Get the value from the user and allocate the memory for the new node dynamically and update the value in the data field of the new node and update its next pointer field to HEAD node.
- Traverse list to the end node and link the next pointer field of the last node to the newnode.

Step 3: Delete

- Get the position of the element to be deleted from the user.
- Traverse the lists to the given position update and link the previous node to the next node of the given position.
- Free the memory allocated for that node.

Step 4: Display

- Print all the elements present in the circular linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>
// Define a node structure
struct Node {
    int data;
    struct Node* next;
};
// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n"); exit(1);
    }
    newNode->data = data;
    newNode->next = NULL; return newNode;
}
// Function to insert a node at the end of the circular linked list
void insertEnd(struct Node** head_ref, int new_data) {
    struct Node* new_node = createNode(new_data);
    if (*head_ref == NULL) {
        *head_ref = new_node;
        new_node->next = *head_ref; // Point to itself for circular
    }
    else {
        struct Node* temp = *head_ref;
        while (temp->next != *head_ref) {
            temp = temp->next;
        }
        temp->next = new_node;
        new_node->next = *head_ref; // Make it circular
    }
}
```

```
}
// Function to print circular linked list
void printList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty\n"); return;
    }
    struct Node* temp = head;
    do {
        printf("%d ", temp->data); temp = temp->next;
    } while (temp != head); printf("\n");
}
// Function to delete a node by key from the circular linked list
void deleteNode(struct Node** head_ref, int key) {
    if (*head_ref == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *head_ref, *prev = NULL;
    // If the node to be deleted is the only node in the list
    if (temp->data == key && temp->next == *head_ref) {
        *head_ref = NULL;
        free(temp);
        return;
    }
    // If the list has more than one node
    if (temp->data == key) {
        while (temp->next != *head_ref) {
            temp = temp->next;
        }
        temp->next = (*head_ref)->next;
        free(*head_ref);
        *head_ref = temp->next;
    } else {
        while (temp->next != *head_ref && temp->data != key) {
            prev = temp;
            temp = temp->next;
        }
        if (temp->data != key) {
            printf("Key not found in the list\n"); return;
        }
        prev->next = temp->next; free(temp);
    }
}

int main() {
    struct Node* head = NULL;
    int data, choice, key;
    do {
        printf("\n1. Insert at End\n2. Delete Node\n3. Print List\n0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data to insert: ");
                scanf("%d", &data);
                insertEnd(&head, data);
                break;
            case 2:
                printf("Enter data to delete: ");
                scanf("%d", &key);
                deleteNode(&head, key);
                break;
```

```

        case 3:
            printf("Circular Linked List: "); printList(head);
            break;
        case 0:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice!\n");
    }
}
while (choice != 0);
// Free memory
struct Node* current = head; if (current != NULL) {
    struct Node* temp; do {
        temp = current;
        current = current->next;
        free(temp);
    } while (current != head);
}
return 0;
}

```

Output:

```

Insert at End
Delete Node
Print List
Exit
Enter your choice: 1 Enter data to insert: 52
Insert at End
Delete Node
Print List
Exit
Enter your choice: 1 Enter data to insert: 56
Insert at End
Delete Node
Print List
Exit
Enter your choice: 1 Enter data to insert: 75
Insert at End
Delete Node
Print List
Exit
Enter your choice: 1 Enter data to insert: 89
Insert at End
Delete Node
Print List
Exit
Enter your choice: 3
Circular Linked List: 52 56 75 89

```

Result:

Thus the C program to perform implementation of circular linked list has been executed and output is verified and also attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.1	2	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 3.1**ARRAY IMPLEMENTATION OF STACK ADT****Aim:**

To write a C program to implement stack ADT using concept of arrays.

Algorithm:

Step 1: Perform push and pop operations

Step 2: Push operation

- For pushing the element into the stack, the element at top of the stack must be greater than or equal to the maximum stack size
- If so, then return stack is full. Then element cannot be pushed into the stack
- Else, increment the top by top+1 and push the element into the stack

Step 3: Pop operation

- To pop an element, the top of the element should be -1.
- Then, return stack is empty and element cannot be popped from stack
- Decrement the top by top-1

Step 4: Display the elements

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100 // Maximum size of the stack
// Stack structure definition
struct Stack {
    int arr[MAX];
    int top;
};
// Function prototypes
void initializeStack(struct Stack *stack);
void push(struct Stack *stack, int element);
int pop(struct Stack *stack);
void displayStack(struct Stack *stack);
int main() {
    struct Stack stack;
    int choice, element;
    initializeStack(&stack);
    while (1) {
        printf("\nStack Operations Menu:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter element to push: ");
                scanf("%d", &element);
                push(&stack, element);
                break;
            case 2:
                pop(&stack);
```

```
        break;
    case 3:
        displayStack(&stack);
        break;
    case 4:
        exit(0);
    default:
        printf("Invalid choice! Please enter a valid option.\n");
    }
}
return 0;
}
// Function to initialize the stack
void initializeStack(struct Stack *stack) {
    stack->top = -1;
}
// Function to push an element onto the stack
void push(struct Stack *stack, int element) {
    if (stack->top == (MAX-1)) {
        printf("Error: Stack overflow. Cannot push element.\n");
        return;
    }
    stack->arr[++stack->top] = element;
    printf("Element pushed: %d\n", element);
}
// Function to pop an element from the stack
int pop(struct Stack *stack) {
    if (stack->top == -1) {
        printf("Error: Stack underflow. Cannot pop element.\n");
    }
    else {
        printf("\nPopped element: %d\n", stack->arr[stack->top--]);
    }
}
// Function to display the stack elements
void displayStack(struct Stack *stack) {
    int i;
    if (stack->top == -1) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements: ");
    for (i = stack->top; i >= 0; i--) {
        printf("%d ", stack->arr[i]);
    }
    printf("\n");
}
```

Output:

Stack Operations Menu:

1. Push

2. Pop

3. Display

4. Exit Enter your choice: 1

Enter element to push: 10

Element pushed: 10

Stack Operations Menu:

1. Push

2. Pop

3. Display

4. Exit Enter your choice: 1

Enter element to push: 20

Element pushed: 20
 Stack Operations Menu:
 1. Push
 2. Pop
 3. Display
 4. Exit
 Enter your choice: 3
 Stack elements: 20 10
 Stack Operations Menu:
 1. Push
 2. Pop
 3. Display
 4. Exit
 Enter your choice: 2
 Popped element: 20
 Stack Operations Menu:
 1. Push
 2. Pop
 3. Display
 4. Exit
 Enter your choice: 3
 Stack elements: 10

Result:

Thus, the C program to implement stack ADT using array has been compiled successfully and output is verified and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.2	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 3.2**ARRAY IMPLEMENTATION OF QUEUE ADT****Aim:**

To write a C program to implement queue ADT using concept of arrays.

Algorithm:

Step 1: Ask the user for the operation like insert, delete, display and exit.

Step 2: According to the option entered, access its respective function using switch statement. Use the variables front and rear to represent the first and last element of the queue.

Step 3: In the function insert(), firstly check if the queue is full. If it is, then print the output as "Queue Overflow". Otherwise take the number to be inserted as input and store it in the variable add_item. Copy the variable add_item to the array queue_array[] and increment the variable rear by 1.

Step 4: In the function delete(), firstly check if the queue is empty. If it is, then print the output as "Queue Underflow". Otherwise print the first element of the array queue_array[] and decrement the variable front by 1.

Step 5: In the function display(), using for loop print all the elements of the array starting from front to rear.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
// Maximum size of the queue
// Queue structure definition
struct Queue {
    int arr[MAX];
    int front;
    int rear;
};
// Function prototypes
void initializeQueue(struct Queue *queue);
int isFull(struct Queue *queue);
int isEmpty(struct Queue *queue);
void enqueue(struct Queue *queue, int element);
int dequeue(struct Queue *queue);
void displayQueue(struct Queue *queue);
int main() {
    struct Queue queue;
    int choice, element;
    initializeQueue(&queue);
    while (1) {
        printf("\nQueue Operations Menu:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter element to enqueue: ");
                scanf("%d", &element);
                enqueue(&queue, element);
                break;
            case 2:
```

```
        element = dequeue(&queue);
        if (element != -1)
            printf("Dequeued element: %d\n", element);
            break;
    case 3:
        displayQueue(&queue);
        break;
    case 4:
        exit(0);
    default:
        printf("Invalid choice! Please enter a valid option.\n");
    }
}
return 0;
}
```

Output:

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element enqueued: 10

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 20

Element enqueued: 20

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 30

Element enqueued: 30

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue elements: 10 20 30

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

Dequeued element: 10

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3
Queue elements: 20 30
Queue Operations Menu:
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4

Result:

Thus the C program to implement queue ADT using array has been compiled successfully and output is verified and satisfied the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.2	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 4.1**LINKED LIST IMPLEMENTATION OF STACK ADT****Aim:**

To write a C program to implement stack using linked list.

Algorithm:**Step 1: Push operation**

- Check if stack is overflow, then print Stack overflow
- Create a new stack node using dynamic memory allocation
i.e. `struct stack * newNode = (struct stack *) malloc(sizeof(struct stack));`
- Assign data to the newly created node using `newNode->data = data;`
- Link new node with the current stack top most element.
- Finally make sure the top of stack should always be the new node

Step 2: Pop operation

- If `size <= 0` then throw stack empty
- Assign the top most element reference to some temporary variable, `struct stack *topNode = top;`. Similarly copy data of stack top element to some variable `int data = top->data;`
- Make second element of stack as top element i.e. `top = top->next;`
- Delete the top most element from memory
- Decrement stack size by one and return data.

Step 3: Display the element**Program:**

```
#include <stdio.h>
#include <stdlib.h>
// Node structure definition
struct Node {
    int data;
    struct Node* next;
};
// Function prototypes
struct Node* createNode(int data);
void push(struct Node** top, int data);
int pop(struct Node** top);
void displayStack(struct Node* top);
int main() {
    struct Node* top = NULL;
    int choice, element;
    while (1) {
        printf("\nStack Operations Menu:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter element to push: ");
                scanf("%d", &element);
```

```
        push(&top, element);
        break;
    case 2:
        element = pop(&top);
        if (element != -1)
            printf("Popped element: %d\n", element);
        break;
    case 3:
        displayStack(top);
        break;
    case 4:
        exit(0);
    default:
        printf("Invalid choice! Please enter a valid option.\n");
    }
}
return 0;
}

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation error\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to push an element onto the stack
void push(struct Node** top, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *top;
    *top = newNode;
    printf("Element pushed: %d\n", data);
}

// Function to pop an element from the stack
int pop(struct Node** top) {
    if (*top == NULL) {
        printf("Error: Stack underflow. Cannot pop element.\n");
        return -1;
    }
    struct Node* temp = *top;
    int poppedElement = temp->data;
    *top = (*top)->next;
    free(temp);
    return poppedElement;
}

// Function to display the stack elements
void displayStack(struct Node* top) {
    if (top == NULL) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements: ");
    struct Node* temp = top;
    while (temp != NULL) {
        printf("%d->", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Output:

Stack Operations Menu:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter element to push: 10

Element pushed: 10

Stack Operations Menu:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter element to push: 20

Element pushed: 20

Stack Operations

Menu:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter element to push: 30

Element pushed: 30

Stack Operations Menu:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 3

Stack elements: 30 20 10

Stack Operations Menu:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 2

Popped element: 30

Stack Operations Menu:

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 3

Stack elements: 20 10

Result:

Thus, the C program to perform stack operation using list has been implemented successfully and the output is verified and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.2	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 4.2

LINKED LIST IMPLEMENTATION OF QUEUE ADT

Aim:

To write a C program to implement queue using linked list.

Algorithm:

- Step 1:** Ask the user for the operations like insert, delete, display.
- Step 2:** According to the entered option access the respective functions.
- Step 3:** Use switch statement to access the functions.
- Step 4:** Use structure with a data and a pointer as the data module.
- Step 5:** Use malloc function to assign the memory dynamically.
- Step 6:** In the enqueue() function ask the user to enter the number to be inserted and store the value into the new data module's data.
- Step 7:** In the dequeue() function delete the element at the front.
- Step 8:** In the displayQueue() function, print the elements in the queue.

Program:

```
#include <stdio.h>
#include <stdlib.h>
// Node structure definition
struct Node {
    int data;
    struct Node* next;
};
// Queue structure definition
struct Queue {
    struct Node* front;
    struct Node* rear;
};
// Function prototypes
struct Node* createNode(int data);
void initializeQueue(struct Queue* queue);
void enqueue(struct Queue* queue, int data);
int dequeue(struct Queue* queue);
void displayQueue(struct Queue* queue);
int main() {
    struct Queue queue;
    int choice, element;
    initializeQueue(&queue);
    while (1) {
        printf("\nQueue Operations Menu:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter element to enqueue: ");
                scanf("%d", &element);
                enqueue(&queue, element);
```

```
        break;
    case 2:
        element = dequeue(&queue);
        if (element != -1)
            printf("Dequeued element: %d\n", element);
        break;
    case 3:
        displayQueue(&queue);
        break;
    case 4:
        exit(0);
    default:
        printf("Invalid choice! Please enter a valid option.\n");
    }
}
return 0;
}
// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation error\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
// Function to initialize the queue
void initializeQueue(struct Queue* queue) {
    queue->front = NULL;
    queue->rear = NULL;
}
// Function to enqueue an element to the queue
void enqueue(struct Queue* queue, int data) {
    struct Node* newNode = createNode(data);
    if (queue->rear == NULL) {
        queue->front = queue->rear = newNode;
        printf("Element enqueued: %d\n", data);
        return;
    }
    queue->rear->next = newNode;
    queue->rear = newNode;
    printf("Element enqueued: %d\n", data);
}
// Function to dequeue an element from the queue
int dequeue(struct Queue* queue) {
    if (queue->front == NULL) {
        printf("Error: Queue underflow. Cannot dequeue element.\n");
        return -1;
    }
    struct Node* temp = queue->front;
    int dequeuedElement = temp->data;
    queue->front = queue->front->next;

    if (queue->front == NULL) {
        queue->rear = NULL;
    }
    free(temp);
}
```

```
        return dequeuedElement;
    }
// Function to display the queue elements
void displayQueue(struct Queue* queue) {
    if (queue->front == NULL) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue elements: ");
    struct Node* temp = queue->front;
    while (temp != NULL) {
        printf("%d->", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Output:

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element enqueued: 10

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 20

Element enqueued: 20

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 30

Element enqueued: 30

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue elements: 10 20 30

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

Dequeued element: 10

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display

4. Exit

Enter your choice: 3

Queue elements: 20 30

Queue Operations Menu:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 4

Result:

Thus the C program to perform linked list implementation of queue has been implemented successfully and the output is verified and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.2	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 5**IMPLEMENTATION OF BINARY SEARCH TREE****Aim:**

To write a C program to implement the binary search tree.

Algorithm:

Step 1: Include the header files

Step 2: Declare a structure with two pointer fields for left child information and right child information.

Step 3: Get the elements one by one and insert it using a insert () function.

- (i) Place the first element as root.
- (ii) Perform the following steps for the next subsequent insertions.
- (iii) If the element is less than one root and its left sub tree is NULL then place it as the left child of the root.
- (iv) Else loop the insertion function taking the first left sub tree element as the root.
- (v) If the element is greater than the root and its right sub tree is NULL then place element as the right child of the root.
- (vi) Else loop the insertion function taking the first right sub tree element as the root.

Step 4: perform search operation to search the elements

Step 5: For deleting any element get the element to be deleted and perform the following steps.

- (i) If the element is leaf node then delete the node.
- (ii) Else if the element has one child deletes it and links its parent node directly with its child node
- (iii) Else if the element has two children then replace with the smallest element of the right sub tree.

Step 6: Display the element of the binary search tree.

Program:

```
#include<stdio.h>
#include<stdlib.h>
struct BST {
    int data;
    struct BST *lchild;
    struct BST *rchild;
};
typedef struct BST * NODE;
NODE create() {
    NODE temp;
    temp = (NODE) malloc(sizeof(struct BST));
    printf("\nEnter The value: ");
    scanf("%d", &temp->data);
    temp->lchild = NULL;
    temp->rchild = NULL;
    return temp;
}
void insert(NODE root, NODE newnode);
```

```
void inorder(NODE root);
void preorder(NODE root);
void postorder(NODE root);
void search(NODE root);
void insert(NODE root, NODE newnode) {
    /*Note: if newnode->data == root->data it will be skipped.    No duplicate nodes are allowed */
    if (newnode->data < root->data) {
        if (root->lchild == NULL;
            root->lchild = newnode;
        else
            insert(root->lchild, newnode);
    }
    if (newnode->data > root->data) {
        if (root->rchild == NULL)
            root->rchild = newnode;
        else
            insert(root->rchild, newnode);
    }
}
}
void search(NODE root) {
    int key;
    NODE cur;
    if(root == NULL) {
        printf("\nBST is empty.");
        return;
    }
    printf("\nEnter Element to be searched: ");
    scanf("%d", &key);
    cur = root;
    while (cur != NULL) {
        if (cur->data == key) {
            printf("\nKey element is present in BST");
            return;
        }
        if (key < cur->data)
            cur = cur->lchild;
        else
            cur = cur->rchild;
    }
    printf("\nKey element is not found in the BST");
}
void inorder(NODE root) {
    if(root != NULL) {
        inorder(root->lchild);

        printf("%d ", root->data);
        inorder(root->rchild);
    }
}
void preorder(NODE root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->lchild);
        preorder(root->rchild);
    }
}
void postorder(NODE root) {
    if (root != NULL) {
        postorder(root->lchild);
        postorder(root->rchild);
    }
}
```

```

        printf("%d ", root->data);
    }
}
void main() {
    int ch, key, val, i, n;
    NODE root = NULL, newnode;
    while(1) {
        printf("\n~~~~~BST MENU~~~~~");
        printf("\n1. Create a BST");
        printf("\n2. Display");
        printf("\n3. Search");
        printf("\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);
        switch(ch) {
            case 1:
                printf("\nEnter the number of elements: ");
                scanf("%d", &n);
                for(i=1;i<=n;i++) {
                    newnode = create();
                    if (root == NULL)
                        root = newnode;
                    else
                        insert(root, newnode);
                }
                break;
            case 2:
                if (root == NULL)
                    printf("\nTree Is Not Created");
                else
                {
                    printf("\nThe Preorder display : ");
                    preorder(root);
                    printf("\nThe Inorder display : ");
                    inorder(root);
                    printf("\nThe Postorder display : ");
                    postorder(root);
                }
                break;
            case 3:
                search(root);
                break;
            case 4:
                exit(0);
        }
    }
}

```

Output:

```

~~~~~BST MENU~~~~~
1. Create a BST
2. Display
3. Search
4. Exit
Enter your choice: 1
Enter the number of elements: 7
Enter The value: 12
Enter The value: 35
Enter The value: 76

```

```

Enter The value: 13
Enter The value: 67
Enter The value: 24
Enter The value: 64
~~~~BST MENU~~~~
1. Create a BST
2. Display
3. Search
4. Exit
Enter your choice: 2
The Preorder display : 12 35 13 24 76 67 64
The Inorder display : 12 13 24 35 64 67 76
The Postorder display : 24 13 64 67 76 35 12
~~~~BST MENU~~~~
1. Create a BST
2. Display
3. Search
4. Exit
Enter your choice: 3
Enter Element to be searched: 64
Key element is present in BST
~~~~BST MENU~~~~
1. Create a BST
2. Display
3. Search
4. Exit
Enter your choice: 3
Enter Element to be searched: 1
Key element is not found in the BST
~~~~BST MENU~~~~
1. Create a BST
2. Display
3. Search
4. Exit
Enter your choice: 4

```

Result:

Thus the C program to perform binary search tree has been implemented successfully and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.3	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 6**IMPLEMENTATION OF AVL TREE****Aim:**

To write a c program to perform the concept of AVL trees.

Algorithm:

Step 1: Check whether the binary search tree is balanced or not

Step 2: Insert a node into binary search tree

Step 3: The tree should be balance after every insertion of node

Step 4: If the node is not balanced, then perform reconstruction

Step 5: Node can be easily removed from the AVL tree

Step 6: Then display the results and stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
// Create Node
struct Node {
    int key;
    struct Node *left;
    struct Node *right;
    int height;
};
// Function prototypes
int max(int a, int b);
int height(struct Node *N);
struct Node *newNode(int key);
struct Node *rightRotate(struct Node *y);
struct Node *leftRotate(struct Node *x);
int getBalance(struct Node *N);
struct Node *insertNode(struct Node *node, int key);
struct Node *minValueNode(struct Node *node);
struct Node *deleteNode(struct Node *root, int key);
void printPreOrder(struct Node *root);
// Utility function to get the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}
// Utility function to get the height of the tree
int height(struct Node *N) {
    if (N == NULL)
        return 0;
    return N->height;
}
// Utility function to create a new node with given key
struct Node *newNode(int key) {
    struct Node *node = (struct Node *) malloc(sizeof(struct Node));
    node->key = key;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    // New node is initially added at leaf
    return node;
}
// Utility function to perform right rotation
struct Node *rightRotate(struct Node *y) {
```

```

struct Node *x = y->left;
struct Node *T2 = x->right;
// Perform rotation
x->right = y;
y->left = T2;
// Update heights
y->height = max(height(y->left), height(y->right)) + 1;
x->height = max(height(x->left), height(x->right)) + 1;
// Return new root
return x;
}
// Utility function to perform left rotation
struct Node *leftRotate(struct Node *x) {
    struct Node *y = x->right;
    struct Node *T2 = y->left;
    // Perform rotation
    y->left = x;
    x->right = T2;
    // Update heights
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;
    // Return new root
    return y;
}
// Utility function to get the balance factor of a node
int getBalance(struct Node *N) {
    if (N == NULL)
        return 0;
    return height(N->left) - height(N->right);
}
// Recursive function to insert a key in the subtree rooted with node
struct Node *insertNode(struct Node *node, int key) {
    // Perform the normal BST insertion
    if (node == NULL)
        return newNode(key);
    if (key < node->key)
        node->left = insertNode(node->left, key);
    else if (key > node->key)
        node->right = insertNode(node->right, key);
    else
        // Equal keys are not allowed in BST
        return node;
    // Update the height of this ancestor node
    node->height = 1 + max(height(node->left), height(node->right));
    // Get the balance factor of this ancestor node to check whether this node became unbalanced
    int balance = getBalance(node);
    // If this node becomes unbalanced, then there are 4 cases
    // Left Left Case
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);
    // Right Right Case
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);
    // Left Right Case
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    // Right Left Case
    if (balance < -1 && key < node->right->key) {

```

```

        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    // Return the (unchanged) node pointer
    return node;
}
// Function to find the node with minimum key value in a subtree rooted with node
struct Node *minValueNode(struct Node *node) {
    struct Node *current = node;
    // Loop down to find the leftmost leaf
    while (current->left != NULL)
        current = current->left;
    return current;
}
// Recursive function to delete a node with given key from the subtree rooted with node
struct Node *deleteNode(struct Node *root, int key) {
    // STEP 1: PERFORM STANDARD BST DELETE
    if (root == NULL)
        return root;
    // If the key to be deleted is smaller than the root's key, then it lies in left subtree
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    // If the key to be deleted is greater than the root's key, then it lies in right subtree
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    // If key is same as root's key, then this is the node to be deleted
    else {
        // Node with only one child or no child
        if ((root->left == NULL) || (root->right == NULL)) {
            struct Node *temp = root->left ? root->left : root->right;
            // No child case
            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else
                // One child case
                *root = *temp;
            // Copy the contents of the non-empty child
            free(temp);
        }
        else {
            // Node with two children: Get the inorder successor (smallest in the right subtree)
            struct Node *temp = minValueNode(root->right);
            // Copy the inorder successor's data to this node
            root->key = temp->key;
            // Delete the inorder successor
            root->right = deleteNode(root->right, temp->key);
        }
    }
}
// If the tree had only one node then return
if (root == NULL)
    return root;
// STEP 2: UPDATE HEIGHT OF THE CURRENT NODE
root->height = 1 + max(height(root->left), height(root->right));
// STEP 3: GET THE BALANCE FACTOR OF THIS NODE (to check whether this node became unbalanced)
int balance = getBalance(root);
// If this node becomes unbalanced, then there are 4 cases
// Left Left Case
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

```

```

// Left Right Case
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
// Right Right Case
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);
// Right Left Case
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
return root;
}
// A utility function to print preorder traversal of the tree
void printPreOrder(struct Node *root) {
    if (root != NULL) {
        printf("%d ", root->key);
        printPreOrder(root->left);
        printPreOrder(root->right);
    }
}
// Main function
int main() {
    struct Node *root = NULL;
    // Insert nodes
    root = insertNode(root, 9);
    root = insertNode(root, 5);
    root = insertNode(root, 10);
    root = insertNode(root, 0);
    root = insertNode(root, 6);
    root = insertNode(root, 11);
    root = insertNode(root, -1);
    root = insertNode(root, 1);
    root = insertNode(root, 2);
    printf("Preorder traversal of the constructed AVL tree is \n");
    printPreOrder(root);
    // Delete nodes
    root = deleteNode(root, 10);
    printf("\n\nPreorder traversal after deletion of 10 \n");
    printPreOrder(root);
    return 0;
}

```

Output:

Preorder traversal of the constructed AVL tree is

9 1 0 -1 5 2 6 10 11

Preorder traversal after deletion of 10

Result:

Thus, the C program to perform AVL tree has been implemented successfully and the output is verified and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.3	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 7.1

BREADTH FIRST SEARCH

Aim:

To write a C program to perform graph traversals using breadth first search.

Algorithm:

Step 1: Create a queue data structure to store nodes to be visited.

Step 2: Enqueue the starting node into the queue.

Step 3: Mark the starting node as visited.

Step 4: While the queue is not empty: ○

- (i) Dequeue a node from the front of the queue.
- (ii) Process the dequeued node (e.g., print its value or perform any desired operation).
- (iii) Enqueue all unvisited neighboring nodes of the dequeued node.
- (iv) Mark each neighboring node as visited to avoid revisiting it.

Step 5: Repeat steps 4 until the queue is empty.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTICES 100
struct Node {
    int vertex;
    struct Node* next;
};
struct Graph {
    struct Node* adjLists[MAX_VERTICES];
    int visited[MAX_VERTICES];
};
struct Queue {
    int items[MAX_VERTICES];
    int front;
    int rear;
};
// Create a node
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}
// Create a graph
struct Graph* createGraph() {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    int i;
    for (i = 0; i < MAX_VERTICES; i++) {
        graph->adjLists[i] = NULL;
        graph->visited[i] = 0;
    }
    return graph;
}
```

```
// Add edge
void addEdge(struct Graph* graph, int src, int dest) {
    // Add edge from src to dest
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;
    // Add edge from dest to src
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

// Create a queue
struct Queue* createQueue() {
    struct Queue* queue = (struct Queue*)malloc(sizeof(struct Queue));
    queue->front = -1;
    queue->rear = -1;
    return queue;
}

// Check if the queue is empty
int isEmpty(struct Queue* queue) {
    return queue->rear == -1;
}

// Add an item to the queue
void enqueue(struct Queue* queue, int value) {
    if (queue->rear == MAX_VERTICES - 1)
        printf("Queue is full\n");
    else {
        if (queue->front == -1)
            queue->front = 0;
        queue->rear++;
        queue->items[queue->rear] = value;
    }
}

// Remove an item from the queue
int dequeue(struct Queue* queue) {
    int item;
    if (isEmpty(queue)) {
        printf("Queue is empty\n");
        item = -1;
    }
    else {
        item = queue->items[queue->front];
        queue->front++;
        if (queue->front > queue->rear) {
            queue->front = queue->rear = -1;
        }
    }
    return item;
}

// Print the graph
void printGraph(struct Graph* graph) {
    int v;
    for (v = 0; v < MAX_VERTICES; v++) {
        struct Node* temp = graph->adjLists[v];
        while (temp) {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("\n");
    }
}
```

```

}
// BFS traversal
void BFS(struct Graph* graph, int startVertex) {
    struct Queue* queue = createQueue();
    graph->visited[startVertex] = 1;
    enqueue(queue, startVertex);
    while (!isEmpty(queue)) {
        int currentVertex = dequeue(queue);
        printf("Visited %d\n", currentVertex);
        struct Node* temp = graph->adjLists[currentVertex];
        while (temp) {
            int adjVertex = temp->vertex;
            if (graph->visited[adjVertex] == 0) {
                graph->visited[adjVertex] = 1;
                enqueue(queue, adjVertex);
            }
            temp = temp->next;
        }
    }
}

int main() {
    struct Graph* graph = createGraph();
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 2);
    addEdge(graph, 1, 2);
    addEdge(graph, 1, 3);
    addEdge(graph, 2, 3);
    printf("Graph:\n");
    printGraph(graph);
    printf("\nBFS Traversal starting from vertex 0:\n");
    BFS(graph, 0);
    return 0;
}

```

Output:

```

2 -> 1 ->
3 -> 2 -> 0 ->
3 -> 1 -> 0 ->
2 -> 1 ->
BFS Traversal starting from vertex 0:
Visited 0
Visited 2
Visited 1
Visited 3

```

Result:

Thus, the C program to perform breadth first search has been implemented successfully. The output for this program is verified and attained the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.4	3	3	2	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 7.2

DEPTH FIRST SEARCH

Aim:

To write a C program to perform graph traversals using depth first search.

Algorithm:

Step 1: Create a stack data structure to store nodes to be visited.

Step 2: Push the starting node onto the stack.

Step 3: Mark the starting node as visited.

Step 4: While the stack is not empty:

- (i) Pop a node from the top of the stack.
- (ii) Process the node (e.g., print its value or perform any desired operation).
- (iii) Push all unvisited neighboring nodes of the popped node onto the stack.
- (iv) Mark each neighboring node as visited to avoid revisiting it.

Step 5: Repeat steps 4 until the stack is empty.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTICES 100
struct Node {
    int vertex;
    struct Node* next;
};
struct Graph {
    struct Node* adjLists[MAX_VERTICES];
    int visited[MAX_VERTICES];
};
// Create a node
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
} // Create a graph
struct Graph* createGraph() {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    int i;
    for (i = 0; i < MAX_VERTICES; i++) {
        graph->adjLists[i] = NULL;

        graph->visited[i] = 0;
    }
    return graph;
}
// Add edge
void addEdge(struct Graph* graph, int src, int dest) {
```

```
// Add edge from src to dest
struct Node* newNode = createNode(dest);
newNode->next = graph->adjLists[src];
graph->adjLists[src] = newNode;
// Add edge from dest to src
newNode = createNode(src);
newNode->next = graph->adjLists[dest];
graph->adjLists[dest] = newNode;
}
// Print the graph
void printGraph(struct Graph* graph) {
    int v;
    for (v = 0; v < MAX_VERTICES; v++)
    {
        struct Node* temp = graph->adjLists[v];
        printf("\n Adjacency list of vertex %d\n ", v);
        while (temp) {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("\n");
    }
}
// DFS traversal
void DFS(struct Graph* graph, int startVertex) {
    struct Node* adjList = graph->adjLists[startVertex];
    struct Node* temp = adjList;
    graph->visited[startVertex] = 1;
    printf("Visited %d \n", startVertex);
    while (temp != NULL) {
        int connectedVertex = temp->vertex;
        if (graph->visited[connectedVertex] == 0) {
            DFS(graph, connectedVertex);
        }
        temp = temp->next;
    }
}
int main() {
    struct Graph* graph = createGraph();
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 2);
    addEdge(graph, 1, 2);
    addEdge(graph, 1, 3);

    addEdge(graph, 2, 3);
    printf("Graph:\n");
    printGraph(graph);
    printf("\nDFS Traversal starting from vertex 0:\n");
    DFS(graph, 0);
    return 0;
}
```

Output:

```
Graph:
2 -> 1 ->
3 -> 2 -> 0 ->
3 -> 1 -> 0 ->
2 -> 1 ->
DFS Traversal starting from vertex 0:
```

Visited 0
 Visited 2
 Visited 3
 Visited 1

Result:

Thus, the C program to perform depth first search has been implemented successfully and the output is verified and also this program satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.4	3	3	2	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 8

APPLICATION OF GRAPHS

Aim:

To write C program to perform Dijkstra Algorithm to find the shortest path.

Algorithm:

Step 1: Start the program

Step 2: Create a cost matrix $C[][]$ from adjacency matrix $adj[][]$. $c[i][j]$ is the cost of going from vertex i to vertex j . If there is no edge between vertices i and j then $C[i][j]$ is infinity.

Step 3: Array visited $[]$ is initialized to zero

Step 4: If the vertex 0 is the source vertex then visited $[0]$ is marked as 1.

Step 5: Created the distance matrix, by storing the cost of vertices from vertex no.0 to $n-1$ from the sources vertex 0.

for ($i=1; i < n; i++$) $distance[i] = cost[0][i];$

Initially, distance of source vertex is taken as 0. i.e $distance[0] = 0;$

Step 6: for($i=1; i < n; i++$)

- choose a vertex w , such that $distance[w]$ is minimum and $visited[w]$ is 0. mark $visited[w]$ as 1.
- Recalculate the shortest distance of remaining vertices from the source.
- Only, the vertices not marked as 1 in array $visited[]$ should be considered for recalculation of distance. i.e for each vertex v
- If ($visited[v] == 0$)
- $Distance[v] = \min(distance[v], Distance[w] + cost[w][v])$

Step 7: Stop the program

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#define MAX_VERTICES 10
#define INFINITY 999
// Function prototypes
void dijkstra(int graph[MAX_VERTICES][MAX_VERTICES], int n, int src);
int main() {
    int graph[MAX_VERTICES][MAX_VERTICES], n, src;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    // Input the adjacency matrix
    printf("Enter the cost adjacency matrix (enter %d for no direct path):\n", INFINITY);
    for (int i = 0; i < n; ++i) {
```

```

        for (int j = 0; j < n; ++j) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == 0 && i != j) {
                graph[i][j] = INFINITY;
                // Assigning infinity where there is no direct edge
            }
        }
    }
    printf("Enter the source vertex (starting from 0): ");
    scanf("%d", &src);
    dijkstra(graph, n, src);
    return 0;
}

void dijkstra(int graph[MAX_VERTICES][MAX_VERTICES], int n, int src) {
    int dist[MAX_VERTICES];
    // Array to store the shortest distance from source to i
    int visited[MAX_VERTICES];
    // Array to mark visited nodes
    int parent[MAX_VERTICES];
    // Array to store the parent of each node in the shortest path tree
    // Initialize all distances as INFINITY and visited[] as false
    for (int i = 0; i < n; ++i) {
        dist[i] = INFINITY;
        visited[i] = 0;
    }
    // Distance of source vertex from itself is always 0
    dist[src] = 0;
    parent[src] = -1;
    // Source has no parent
    // Find shortest path for all vertices
    for (int count = 0; count < n - 1; ++count) {
        // Pick the minimum distance vertex from the set of vertices not yet processed
        int u = -1;
        for (int v = 0; v < n; ++v) {
            if (!visited[v] && (u == -1 || dist[v] < dist[u])) {
                u = v;
            }
        }
        // Mark the picked vertex as visited
        visited[u] = 1;
        // Update dist value of the adjacent vertices of the picked vertex
        for (int v = 0; v < n; ++v) {
            if (!visited[v] && graph[u][v] && dist[u] != INFINITY && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
    }
    // Print the shortest paths
    printf("\nShortest Paths from Source Vertex %d:\n", src);
    for (int i = 0; i < n; ++i) {
        if (i != src) {
            printf("Path to vertex %d: Cost = %d | Path = %d", i, dist[i], i);
            int p = parent[i];
            while (p != -1) {
                printf(" <- %d", p);
                p = parent[p];
            }
            printf("\n");
        }
    }
}

```

```
}
}
```

Output:

Enter the number of vertices: 5
 Enter the cost adjacency matrix (enter 999 for no direct path):
 0 10 5 999 999
 999 0 2 1 999
 999 3 0 9 2
 4 999 999 0 7
 999 999 999 6 0
 Enter the source vertex (starting from 0): 0
 Shortest Paths from Source Vertex 0:
 Path to vertex 1: Cost = 8 | Path = 1 <- 2 <- 0
 Path to vertex 2: Cost = 5 | Path = 2 <- 0
 Path to vertex 3: Cost = 9 | Path = 3 <- 1 <- 2 <- 0
 Path to vertex 4: Cost = 7 | Path = 4 <- 2 <- 0

Result:

Thus the C program for Graph has been implemented successfully and the output is verified. This program satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.4	3	3	2	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 9**IMPLEMENTATION OF PRIORITY QUEUES****Aim:**

To write C program to implement priority queues.

Algorithm:**Step 1: Initialization:**

- Create a new heap structure with the desired capacity.

Step 2: Insert:

- Add the element to the end of the heap array.
- Compare the inserted element with its parent. If the parent is greater, swap them.
- Repeat the comparison and swapping process until the min-heap property is restored.

Step 3: Delete Min:

- Replace the root (minimum element) with the last element in the heap.
- Remove the last element.
- Compare the new root with its children and swap it with the smaller child if necessary.
- Repeat the comparison and swapping process until the min-heap property is restored.

Step 4: Display:

- Print the elements of the heap in any desired order.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct heap {
    int c;
    // capacity
    int s;
    // size
    int *element;
};

typedef struct heap *pqueue;
pqueue initialize(int max) {
    pqueue heap1;
    if (max <= 3) {
        printf("\n Priority queue is too small\n");
        exit(EXIT_FAILURE);
    }
    heap1 = (pqueue)malloc(sizeof(struct heap));
    if (heap1 == NULL) {
        printf("\n Out of space\n");
        exit(EXIT_FAILURE);
    }
    heap1->element = (int *)malloc((max + 1) * sizeof(int));
    if (heap1->element == NULL) {
        printf("\n Out of space\n");
        exit(EXIT_FAILURE);
    }
    heap1->c = max;
    heap1->s = 0;
    heap1->element[0] = 0;
    return heap1;
}
```

```
int isempty(pqueue heap1) {
    return (heap1->s == 0);
}

int isfull(pqueue heap1) {
    return (heap1->s == heap1->c);
}

void insert(int x, pqueue heap1) {
    int i;
    if (isfull(heap1)) {
        printf("\n Heap is full\n");
        return;
    }
    for (i = ++heap1->s; heap1->element[i / 2] > x; i = i / 2)
        heap1->element[i] = heap1->element[i / 2];
    heap1->element[i] = x;
}

int deletemin(pqueue heap1) {
    int i, child, min, last;
    if (isempty(heap1)) {
        printf("\n Heap is empty\n");
        return heap1->element[0];
    }
    min = heap1->element[1];
    last = heap1->element[heap1->s--];
    for (i = 1; i * 2 <= heap1->s; i = child) {
        child = i * 2;
        if (child != heap1->s && heap1->element[child + 1] < heap1->element[child])
            child++;
        if (last > heap1->element[child])
            heap1->element[i] = heap1->element[child];
    }
    else
        break;
    }
    heap1->element[i] = last;
    return min;
}

void display(pqueue heap1) {
    int i;
    printf("\nElements in the heap: ");
    for (i = 1; i <= heap1->s; i++) {
        printf("%d ", heap1->element[i]);
    }
    printf("\n");
}

int main() {
    int s, ch, ele;
    pqueue heap1;
    printf("\nEnter the size of the queue: ");
    scanf("%d", &s);
    heap1 = initialize(s);
    do {
        printf("\nMenu\n1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
```

```
        case 1:
            printf("\nEnter the element to insert: ");
            scanf("%d", &ele);
            insert(ele, heap1);
            break;
        case 2:
            ele = deletemin(heap1);
            printf("\nThe deleted element is %d\n", ele);
            break;
        case 3:
            display(heap1);
            break;
        case 4:
            printf("\nExiting...\n");
            break;
        default:
            printf("\nInvalid choice\n");
            }
    }
    while (ch != 4);
    free(heap1->element);
    free(heap1);
    return 0;
}
```

Output:

Enter the size of the queue: 5
Menu
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: 87

Menu
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: 3

Menu
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: 65

Menu
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Elements in the heap: 3 87 65

Menu

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

The deleted element is 3

Menu

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Elements in the heap: 65 87

Result:

Thus, the C program to implement priority queue has been executed successfully and output is verified and also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.3	3	2	1	1	2	-	1	3	-	-	3	-	3	-

EX.NO: 10.1**IMPLEMENTATION OF LINEAR SEARCH****Aim:**

To write C program to implement the linear search.

Algorithm:

Step 1: Start with the first item in the list.

Step 2: Compare the current item to the target.

Step 3: If the current value matches the target then we declare victory and stop.

Step 4: If the current value is less than the target then set the current item to be the next item and repeat from 2.

Program:

```
#include <stdio.h>
// Function for linear search
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; ++i) {
        if (arr[i] == key) {
            return i;
            // Return the index if element is found
        }
    }
    // Return -1 if element is not found
    return -1;
}

int main() {
    int arr[] = {12, 45, 67, 23, 56, 89, 9, 43};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key, index;
    printf("\n—Linear Search--\n");
    printf("Enter the element to search: ");
    scanf("%d", &key);
    index = linearSearch(arr, n, key);
    if (index != -1) {
        printf("Element %d found at position %d.\n", key, index + 1);
    }
    else {
        printf("Element %d not found in the array.\n", key);
    }
    return 0;
}
```

Output:

—Linear Search--

Enter the element to search: 45

Element 45 found at position 2.

—Linear Search--

Enter the element to search: 11

Element 11 not found in the array.

Result:

Thus, the C program for Linear Searching has been implemented successfully and the output is verified and also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 10.2**IMPLEMENTATION OF BINARY SEARCH****Aim:**

To write a C program to perform binary search.

Algorithm:

Step 1: Start the program

Step 2: Read n numbers and search value.

Step 3: If search value is equal to middle element then print value is found.

Step 4: If search value is less than middle element then search left half of list with the same method.
Else search right half of list with the same method.

Step 5: Display the result

Step 6: Stop the program

Program:

```
#include <stdio.h>
int main() {
    int n, i, search, f = 0, low, high, mid, a[20];
    printf("\n***Binary Search***\n");
    printf("\nEnter the number of elements: ");
    scanf("%d", &n);
    // Input array elements in ascending order
    printf("Enter %d numbers in ascending order:\n", n);
    for (i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }
    printf("Enter the search element: ");
    scanf("%d", &search);
    low = 0;
    high = n - 1;
    // Binary search
    while (low <= high) {
        mid = (low + high) / 2;
        if (search < a[mid])
            high = mid - 1;
        else if (search > a[mid])
            low = mid + 1;
        else {
            f = 1;
            printf("Element %d found at index %d.\n", search, mid);
            break;
        }
    }

    if (f == 0)
        printf("Element %d is not present in the array.\n", search);
}
```

```
    return 0;
}
```

Output:

Binary Search

Enter the number of elements: 4

Enter 4 numbers in ascending order:

a[0] = 23

a[1] = 45

a[2] = 67

a[3] = 87

Enter the search element: 45

Element 45 found at index 1.

Binary Search

Enter the number of elements: 4

Enter 4 numbers in ascending order:

a[0] = 56

a[1] = 78

a[2] = 87

a[3] = 90

Enter the search element: 10

Element 10 is not present in the array.

Result:

Thus the C program to perform binary search has been implemented successfully and also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 10.3**INSERTION SORT****Aim:**

To write a C program to perform insertion sort on the given set of values.

Algorithm:

Step 1: Get the number of elements and the array elements from the user.

Step 2: Compare the current index element in an array with all the previous index elements present in an array. If the current index value is less than the previous index values then swap the numbers.

Step 3: Repeat the step 2 for all the elements in an array. Now the array is in sorted order.

Step 4: Display the sorted array.

Program:

```
#include <stdio.h>
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int choice, n, i;
    printf("\n-----INSERTION SORT      \n ");
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements are :\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    do {
        printf("\nMenu:\n");
        printf("1. Sort using Insertion Sort\n");
        printf("2. Print the sorted array\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice); switch (choice) {
            case 1:
                insertionSort(arr, n);
```

```

        printf("Array sorted using Insertion Sort.\n");
        break;
    case 2:
        printf("Sorted array: ");
        printArray(arr, n);
        break;
    case 3:
        printf("Exiting the program.\n");
        break;
    default:
        printf("Invalid choice! Please enter a valid menu option.\n");
    }
} while (choice != 3);
return 0;
}

```

Output:

-----INSERTION SORT

Enter the number of elements: 4

Enter 4 elements are :

1
56
23
78

Menu:

1. Sort using Insertion Sort
2. Print the sorted array
3. Exit

Enter your choice: 1

Array sorted using Insertion Sort.

Menu:

1. Sort using Insertion Sort
2. Print the sorted array
3. Exit

Result:

Thus, the C program to perform insertion sort has been implemented successfully and satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 10.4**QUICK SORT****Aim:**

To write a C program to implement Quick sort for given set of elements.

Algorithm:**Step 1: Partitioning:**

- (i) Choose a pivot element from the array (typically the last element).
- (ii) Reorder the array such that all elements less than the pivot are moved to its left, and all elements greater than the pivot are moved to its right.
- (iii) The pivot is now in its correct sorted position.

Step 2: Recursion:

- (i) Recursively apply the above steps to the sub-arrays of elements with smaller and greater values than the pivot.

Program:

```
#include<stdio.h>
// Function to swap two elements in the array
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
// Function to partition the array and return the index of the pivot
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    // Choosing the last element as the pivot
    int i = low - 1;
    // Index of smaller element
    for (int j = low; j < high; j++) {
        // If current element is smaller than the pivot
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    // Place the pivot element in its correct position
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Function to implement Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // Partitioning index
        int pi = partition(arr, low, high);
        // Recursive sorting of the sub-arrays
```

```

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
// Function to print the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);    printf("\n");
}
int main() {
    int arr[] = {12, 7, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    quickSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    printArray(arr, n);
    return 0;
}

```

Output:

Original array: 12 7 11 13 5 6
 Sorted array: 5 6 7 11 12 13

Result:

Thus, the C program to perform quick sort has been implemented successfully and also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 11.1**IMPLEMENTATION OF HASING WITH SEPARATE CHAINING****Aim:**

To write a C program to perform the concept of hashing-separate chaining method.

Algorithm:

Step 1: Start the program

Step 2: Read the value to be inserted

Step 3: Create a new node using malloc function

Step 4: Assign the value read to the data field of newnode (newnode -> data =value) compute the index
index = value % TABLE_SIZE if head[index] is NULL then attach the newnode as first node else
attach the newnode as last node.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define TABLE_SIZE 3
struct node {
    int data;
    struct node *next;
};
struct node *head[TABLE_SIZE] = {NULL};
void insert(int val) {
    int i = val % TABLE_SIZE;
    struct node *newnode = (struct node *)malloc(sizeof(struct node));
    if (newnode == NULL) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }
    newnode->data = val;
    newnode->next = NULL;
    if (head[i] == NULL) {
        head[i] = newnode;
    }
    else {
        struct node *c = head[i];
        while (c->next != NULL) {
            c = c->next;
        }
        c->next = newnode;
    }
}
void display() {
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf("Entries at index %d: ", i);
        struct node *temp = head[i];
        if (temp == NULL) {
            printf("No Hash Entry\n");
        }
        else {
            while (temp != NULL) {
                printf("%d -> ", temp->data);
                temp = temp->next;
            }
            printf("NULL\n");
        }
    }
}
```

```

    }
}
int main() {
    int opt, val;
    while (1) {
        printf("\nPress 1. Insert\t 2. Display \t3. Exit \n");
        scanf("%d", &opt);
        switch (opt) {
            case 1:
                printf("\nEnter a value to insert into hash table: ");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                display();
                break;
            case 3:
                exit(0);
            default:
                printf("\nInvalid option\n");
        }
    }
    return 0;
}

```

Output:

```

Press 1. Insert   2. Display  3. Exit
1
Enter a value to insert into hash table: 12
Press 1. Insert 2. Display  3. Exit
1
Enter a value to insert into hash table: 7
Press 1. Insert 2.Display  3. Exit
1
Enter a value to insert into hash table: 3
Press 1. Insert 2. Display  3. Exit
2
Entries at index 0: No Hash
Entry Entries at index 1: 12 -> NULL
Entries at index 2: 3 -> 7 -> NULL
Press 1. Insert 2. Display  3. Exit
1
Enter a value to insert into hash table: 15
Press 1. Insert 2. Display  3. Exit
2
Entries at index 0: No Hash Entry
Entries at index 1: 12 -> NULL
Entries at index 2: 3 -> 7 -> 15 -> NULL
Press 1. Insert 2. Display  3. Exit
3

```

Result:

Thus, the C program to perform hashing technique (separate chaining) has been implemented successfully and the output is verified. Also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 11.2

IMPLEMENTATION OF HASING WITH OPEN ADDRESSING

Aim:

To write a C program to implement the concept of hashing using linear probing in open addressing.

Algorithm:

Step 1: Create hash table and hash function.

Step 2: Assigning INT_MIN indicates that cell is empty.

Step 3: INT_MIN and INT_MAX indicates that cell is empty.

Step 4: If cell is empty loop will break and go to bottom of the loop to insert element.

Step 5: If table is full we should break, if not check this, loop will go to infinite loop.

Step 6: If the hash element is empty, the deletion has immediately failed.

Step 7: Otherwise, check for a match between the target and data key.

Step 8: If there is a match, delete the hash element (and set the flag).

Step 9: If there is no match, probe the table until either:

- (i) An match is found between the target key and the data's key;
- (ii) The data can be deleted, and the deleted flag set.
- (iii) A completely empty hash element is found.

Step 10: If the hash element is empty, the search has immediately failed.

Step 11: Otherwise, check for a match between the search and data key.

Step 12: If there is a match, return the data.

Step 13: If there is no match, probe the table until either:

- (i) An match is found between the search and data key.
- (ii) A completely empty hash element is found.

Program:

```
#include<stdio.h>
#include<limits.h>
void insert(int ary[], int hFn, int size) {
    int element, pos, n = 0;
    printf("Enter key element to insert\n");
    scanf("%d", &element);
    pos = element % hFn;
    while (ary[pos] != INT_MIN)
    {
        // INT_MIN and INT_MAX indicate that cell is empty or deleted
        if (ary[pos] == INT_MAX)
            break;
        pos = (pos + 1) % hFn;
        n++;
        if (n == size) {
            printf("Hash table is full. No place to insert this element.\n");
            return;
        }
    }
    ary[pos] = element;
}
void delete(int ary[], int hFn, int size) {
    int element, pos, n = 0;
```

```
printf("Enter element to delete\n");
scanf("%d", &element);
pos = element % hFn;
while (n++ != size) {
    if (ary[pos] == INT_MIN) {
        printf("Element not found in hash table\n");
        return;
    }
    else if (ary[pos] == element) {
        ary[pos] = INT_MAX; // Mark as deleted
        printf("Element deleted\n");
        return;
    }
    else {
        pos = (pos + 1) % hFn;
    }
}
printf("Element not found in hash table\n");
}

void search(int ary[], int hFn, int size) {
int element, pos, n = 0;
printf("Enter element you want to search\n");
scanf("%d", &element);
pos = element % hFn;
while (n++ != size) {
    if (ary[pos] == element) {
        printf("Element found at index %d\n", pos);
        return;
    }
    else if (ary[pos] == INT_MAX || ary[pos] != INT_MIN) {
        pos = (pos + 1) % hFn;
    }
}
printf("Element not found in hash table\n");
}

void display(int ary[], int size) {
printf("Index\tValue\n");
for (int i = 0; i < size; i++) {
    printf("%d\t", i);
    if (ary[i] == INT_MIN)
    {
        printf("Empty\n");
    }
    else if (ary[i] == INT_MAX) {
        printf("Deleted\n");
    }
    else
    {
        printf("%d\n", ary[i]);
    }
}
}

int main() {
int size, hFn, choice;
printf("Enter size of hash table\n");
scanf("%d", &size);
int ary[size];
printf("Enter hash function (if mod 10, enter 10)\n");
scanf("%d", &hFn);
for (int i = 0; i < size; i++)
```

```
    ary[i] = INT_MIN;
    // Assigning INT_MIN indicates that cell is empty
    do {
        printf("\nEnter your choice\n");
        printf("1 -> Insert\n2 -> Delete\n3 -> Display\n4 -> Search\n0 -> Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                insert(ary, hFn, size);
                break;
            case 2:
                delete(ary, hFn, size);
                break;
            case 3:
                display(ary, size);
                break;
            case 4:
                search(ary, hFn, size);
                break;
            case 0:
                printf("Exiting...\n");
                break;
            default:
                printf("Enter correct choice\n");
                break;
        }
    } while (choice != 0);
    return 0;
}
```

Output:

Enter size of hash table

5

Enter hash function (if mod 10, enter 10)

5

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search

0 -> Exit

1

Enter key element to insert 12

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search

0 -> Exit

1

Enter key element to insert 25

Enter your choice 1 -> Insert 2 -> Delete 3 -> Display 4 -> Search 0 -> Exit 1

Enter key element to insert 20

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search

0 -> Exit

1

Enter key element to insert 8

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search 0 -> Exit

1

Enter key element to insert 18

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search

0 -> Exit

3

Index Value

0 25

1 Empty

2 20

3 8

4 18

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search

0 -> Exit

2

Enter element to delete

8

Element deleted

Enter your choice

1 -> Insert

2 -> Delete

3 -> Display

4 -> Search

0 -> Exit

3

Index Value

0 25

1 Empty

2 20

3 Deleted

4 18

Result:

Thus the C program to perform hashing technique (open addressing) has been implemented successfully and the output is verified. Also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-

EX.NO: 12

MINI PROJECT: STUDENT RECORD SYSTEM

Aim:

To write a C program to implement Student record system using data structures concepts.

Algorithm:

Step 1: Add Student Details: Get data from user and add a student to the list of students. While adding the students into the list, check for the uniqueness of the roll number.

Step 2: Find the student by the given roll number: This function is to find the student record for the given roll number and print the details.

Step 3: Find the student by the given first name: This function is to find all the students with the given first name and print their details.

Step 4: Find the students registered in a course: This function is to find all the students who have registered for a given course.

Step 5: Count of Students: This function is to print the total number of students in the system

Step 6: Delete a student: This function is to delete the student record for the given roll number.

Step 7: Update Student: This function is to update the student records. This function does not ask for new details for all fields but the user should be able to pick and choose what he wants to update.

Program:

```
#include<stdio.h>
#define SIZE 50
struct Student {
    char name[30];
    int rollno;
    int sub[3];
};
void main() {
    int i, j, max, count, total, n, a[SIZE], ni;
    struct Student st[SIZE];
    printf("Enter how many students: ");
    scanf("%d", &n);
    /* for loop to read the names and roll numbers*/
    for (i = 0; i < n; i++) {
        printf("\nEnter name and roll number for student %d : ", i);
        scanf("%s", &st[i].name);
        scanf("%d", &st[i].rollno);
    }
    /* for loop to read ith student's jth subject*/
    for (i = 0; i < n; i++) {
        for (j = 0; j <= 2; j++) {
            printf("\nEnter marks of student %d for subject %d : ", i, j);
            scanf("%d", &st[i].sub[j]);
        }
    }
    /* (i) for loop to calculate total marks obtained by each student*/
    for (i = 0; i < n; i++) {
        total = 0;
        for (j = 0; j < 3; j++) {
            total = total + st[i].sub[j];
        }
    }
}
```

```

        printf("\nTotal marks obtained by student %s are %d", st[i].name, total);
        a[i] = total;
    }
    /* (ii) for loop to list out the student's roll numbers who have secured the highest marks in each subject */
    roll number who secured the highest marks */
    for(j = 0; j < 3; j++) {
        max = 0;
        for (i = 0; i < n; i++) {

            if (max < st[i].sub[j]) {
                max = st[i].sub[j];
                ni = i;
            }
        }
        printf("\nStudent %s got maximum marks = %d in Subject : %d", st[ni].name, max, j);
    }
    max = 0;
    for (i = 0; i < n; i++) {
        if (max < a[i]) {
            max = a[i];
            ni = i;
        }
    }
    printf("\n%s obtained the total highest marks.", st[ni].name);
    getch();
}

```

Output:

Enter how many students: 2
 Enter name and roll number for student 0 : ARUN
 4211101
 Enter name and roll number for student 1 : KUMAR
 4211102
 Enter marks of student 0 for subject 0 : 78
 Enter marks of student 0 for subject 1 : 98
 Enter marks of student 0 for subject 2 : 67
 Enter marks of student 1 for subject 0 : 78
 Enter marks of student 1 for subject 1 : 55
 Enter marks of student 1 for subject 2 : 57
 Total marks obtained by student ARUN are 243
 Total marks obtained by student KUMAR are 190
 Student ARUN got maximum marks = 78 in Subject : 0
 Student ARUN got maximum marks = 98 in Subject : 1
 Student ARUN got maximum marks = 67 in Subject : 2
 ARUN obtained the total highest marks.

Result:

Thus, the C program to create student record has been implemented successfully and the output is verified. Also satisfies the following PO's and PSO's.

CO/PO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
L203.5	3	3	3	2	2	-	1	3	-	-	3	-	3	-